

哈尔滨工业大学

编译系统 2022 春

实验一

学院:	未来技术学院
姓名:	郭庆吉
学号:	2021113250
指导教师:	朱庆福

一、实验目的

1. 巩固对词法分析与语法分析的基本功能和原理的认识
2. 能够应用自动机的知识进行词法与语法分析
3. 理解并处理词法分析与语法分析中的异常和错误

二、实验环境

1. Ubuntu 20.04
2. GCC 11.4
3. Flex 2.6.4
4. Bison 3.8.2

三、实验过程

设计数据结构

本次实验中，我们使用语法分析树进行语法分析。这意味着我们需要使用一种树形结构来表示数据。为了尽量节省空间，我们选择了多叉树的二叉树表示法，其中左子节点代表子节点，右子节点代表兄弟节点。这种表示方法允许我们使用链表存储树结构，同时确切地了解树的大小，而不会浪费额外的存储空间。每个树节点对应我们语法分析中的一个语法单元。我们使用一个联合体来表示树节点的信息，并通过这种方式完成了数据结构的设计。

由于我们采用链表来尽量减少无效空间的使用，因此无法事先确定特定语法单元的子节点数量。因此，我们需要使用 stdarg 库中的不定参数量的 va_list 数据结构来存储和处理子节点。

在确保代码正确无误后，我们需要输出整段代码。输出的过程本质上是对语法分析树进行遍历。因此，我们需要在树结构的基础上实现树的深度优先遍历。这样，我们就完成了数据结构的设计。

```
struct Parsetree
{
    int line;
    char *Token;
    int isleaf;
    union
    {
        char* Id_Type;
        int intval;
        float floatval;
    };
    struct Parsetree *firstchild,*nxtbro;
};
```

```
static void dfs(Treenode now, int depth)
{
    if (now==NULL||now->Token==NULL) return;
    printf("%s",2*depth,"");
    printf("%s",now->Token);
    if(!strcmp(now->Token,"ID")||!strcmp(now->Token,"TYPE"))
        printf(": %s",now->Id_Type);
    else if(!strcmp(now->Token,"INT"))
        printf(": %d",now->intval);
    else if(!strcmp(now->Token,"FLOAT"))
        printf(": %f",now->floatval);
    else if(!now->isleaf)
        printf(" (%d)",now->line);
    puts("");
    dfs(now->firstchild,depth+1);
    dfs(now->nxtbro,depth);
}
```

基于 flex 的词法分析

我们通过 lexical.l 文件的编写实现对于词法的分析,在这个过程中比较重要的一步就是书写特定词的正则表达式。较为复杂的三个部分分别是 INT, FLOAT 和 ID。对于其它的部分,我们只需要将名称与对应的符号进行匹配即可。在最终的词法分析中,我们会将 C-程序拆分成一个一个的词进行输出。

```
INT8 [+~]?0[1-7][0-7]*
INT10 [+~]?(<0|[1-9][0-9]*)
INT16 [+~]?0[Xx][0-9a-fA-F]+
INT {INT8}|{INT10}|{INT16}

FLOAT ({INT10}\.[0-9]+)|{[+~]?([0-9]*\.[0-9]+|[0-9]+\.[0-9]*)[Ee][+~]?[0-9]+)

NUMBER {INT}|{FLOAT}

ID [a-zA-Z_][a-zA-Z0-9_]*
```

我们同样也会针对一些特定的错误词法设计出相对应的错误解决办法,在本实验中我们将这种错误记为了 A 类错误。比如八进制数首位为 0 但是后面的数中不能出现 9,所以对首位为 0 且后面出现 9 的数定义为 INT8_ERROR。同理,对于 INT16_ERROR 我们认定在 0x 后面出现 [g-zG-Z] 都为错误。对于指数形式的浮点数,我们也按其形式定义。

```
INT8_ERROR 0[0-7]*[8-9]+[0-7]*
INT16_ERROR 0[Xx][a-fA-F0-9]*[g-zG-Z]+[a-fA-F0-9]*
FLOAT_ERROR [0]+(<0|[1-9][0-9]*)\.[0-9]+|\. [0-9]+|([0-9]+\.[0-9]+[Ee][+~]?[0-9]*|([+~]?[0-9]*\.[0-9]+|[0-9]+\.[0-9]*)[Ee][+~]?[0-9]+\.[0-9]+)
NUM_ERROR {INT8_ERROR}|{INT16_ERROR}|{FLOAT_ERROR}
ID_ERROR [0-9]+[a-zA-Z_]+
```

对于注释符号,遇到//或者/*我们一直读入字符直到\n 或者*/。

基于 bison 的语法分析

我们通过 syntax.y 文件的编写实现对于语法的分析,在这个过程中比较重要的一步就是错误的匹配。我们会将各种正确情况进行枚举,然后在 syntax.y 代码中列举正确的情况的后续操作,而不符合正确情况的我们就统一使用 error 单元进行匹配。当然在做这些之前我们先要规定整个语法分析过程中的终结符与非终结符,因为我们的语法分析是基于自动机的知识进行实现的。

```
%type <node> Program ExtDefList ExtDef ExtDeclList // High-level Definitions
%type <node> Specifier StructSpecifier OptTag Tag // Specifiers
%type <node> VarDec FunDec VarList ParamDec // Declarators
%type <node> CompSt StmtList Stmt // Statements
%type <node> DefList Def Dec Declist // Local Definitions
%type <node> Exp Args // Expressions
```

我们每当处理到一个合法的节点,首先我们需要为其声明一个树节点,而后我们需要将这个节点的信息以及这个节点的子节点的数量以及编号全部传入声明节点的函数。我们判断终结符的方式是判断当前节点的子节点数量是否等于 0,若为 0 则是,反之则不是。对于终结符我们需要提取其信息,对于非终结符则无需提取信息,只需向下继续处理子节点即可。处理节点这部分代码体现在 treenode.h 中如下:

```
if (amount==0)
{
    rt->isleaf=1;
    rt->firstchild=NULL;
    if(!strcmp(TOKEN,"ID"))||(!strcmp(TOKEN,"TYPE"))
    {
        char *str;
        str=(char*)malloc(sizeof(char)*40);
        strcpy(str,yytext);
        rt->Id_Type=str;
    }
    else if(!strcmp(TOKEN,"INT"))
        rt->intval=atoi(yytext);
    else if(!strcmp(TOKEN,"FLOAT"))
        rt->floatval=atof(yytext);
}

else
{
    va_list list;//x0x1x2x3x4x5x6x7x8x9xaxbxcx
    va_start(list,amount);//μ0μ1μ2μ3μ4μ5μ6μ7μ8μ9μαμβμγμδμεμζμημθμi
    ch=va_arg(list,Treenode);
    rt->firstchild=ch;
    if (amount!=1)
    {
        for (int i=1;i<=amount-1;i++)
        {
            ch->nxttbro=va_arg(list,Treenode);
            ch=ch->nxttbro;
        }
        ch->nxttbro = NULL;
    }
    va_end(list);
}
return rt;
```

实验操作流程

(1) 编写 lexical.l 代码,利用 flex 进行编译得到 lex.yy.c

- (2) 编写主函数 main.c 与 lex.yy.c 共同编译得到词法分析工具 scanner
- (3) 编写 syntax.y 代码，在其中引用 lex.yy.c，利用 bison 进行编译得到 syntax.tab.c 和 syntax.tab.h
- (4) 在 lexical.l 中引用 syntax.tab.h 重新编译得到新的 lex.yy.c
- (5) 编写主函数 main.c，与 syntax.tab.c 共同编译得到语法分析工具 parser

实验结果（仅展示部分）

```

gqj@gqj-virtual-machine:/mnt/hgfs/ubuntu共享/代码$ ./parser testcase_1
error type A at line 1: mysterious character '#'
Error type B at Line 1: syntax error.
gqj@gqj-virtual-machine:/mnt/hgfs/ubuntu共享/代码$ ./parser testcase_2
error type A at line 4: mysterious character '%'
Error type B at Line 4: syntax error.

```

<pre> gqj@gqj-virtual-machine:/mnt/hgfs/ubuntu共享/代码\$./parser testcase_13 Program(1) ExtDefList(1) ExtDef(1) Specifier(1) TYPE: int FunDec(1) ID: fibonaccl LP VarList(1) ParamDec(1) Specifier(1) TYPE: int VarDec(1) ID: n RP CompSt(2) LC DefList(3) Def(3) Specifier(3) TYPE: int DecList(3) Dec(3) VarDec(3) </pre>	<pre> ID: a ASSIGNOP Exp(3) INT: 0 COMMA DecList(3) Dec(3) VarDec(3) ID: b ASSIGNOP Exp(3) INT: 1 COMMA DecList(3) Dec(3) VarDec(3) ID: l ASSIGNOP Exp(3) INT: 0 SEMI StmtList(5) Stmt(5) WHILE LP </pre>	<pre> Exp(5) Exp(5) ID: i RELOP Exp(5) ID: n RP Stmt(6) CompSt(6) LC StmtList(8) Stmt(8) Exp(8) ID: write LP Args(8) Exp(8) ID: b RP SEMI RC </pre>
---	---	---

四、实验心得与体会

1. 通过本次实验，我学会了借助 flex, bison 等工具完成对于词法和语法正确性的分析
2. 在实验的过程中，我巩固了自动机这门课的相关知识，并将其应用至本次实验中，利用其对于结果做出了正确的分析。
3. 在实验中，我也巩固了对于正则表达式的理解，对于 C++ 语言中给定的词法和文法利用正则表达式做出了正确的表示。